

Desarrollo del Taller Practico

Integrantes: Lenin Barrios, Daniela Calderon, Alberto Marquez, Andres Garcia, Jose Barbosa

Actividad 1: El Tablero de Tareas del Curso Kanban

Objetivo: Visualizar el trabajo pendiente de la materia.

Instrucciones: En grupos, diseñen un tablero Kanban en una pizarra o herramienta digital. Creen columnas como "Tareas Pendientes", "En Progreso (Estudiando/Haciendo)", "Entregado" y "Calificado". Añadan tarjetas para cada tarea, proyecto o examen del curso. Discutan: ¿Dónde está el WIP? ¿Hay cuellos de botella?

Desarrollo:

- **Enlace del tablero:**

<https://trello.com/invite/b/68f24d0a0508d2f6326e8caf/ATTI7afd8206b19e45d47ed52a1f365b8208A280F295/tareas>

- **¿Qué es el WIP?**

R/ WIP significa "Work In Progress" (Trabajo en Progreso), que se refiere a todas las tareas que están en proceso pero que aún no se han completado.

- **¿Dónde está el WIP (Trabajo en Progreso)?**

R/ El WIP se encuentra en la columna "En Progreso (Estudiando/Haciendo)", donde se encuentran las tareas que están en proceso actualmente, como "Habilidades y competencias del trabajo colaborativo."

- **¿Hay cuellos de botella?**

R/ En este caso, parece que hay un pequeño cuello de botella en la columna de "Tareas Pendientes." Aunque hay varias tareas en esa columna, como el "Proyecto final" y el "Momento evaluativo #2," ninguna de estas tareas se ha movido a la siguiente fase. Debido a que ya están informadas, pero no hay como tal un deber que hacer, a la espera de que se tendrá que realizar por parte de la profesora.

Actividad 2: Simulación de un Sprint Planning Scrum

Objetivo: Planificar un Sprint para un proyecto ficticio.

Proyecto: "Desarrollar una app para organizar torneos de videojuegos".

Instrucciones: Asignen roles (Product Owner, Scrum Master, Developers). El PO presenta un Product Backlog simple (ej. "Registro de usuario", "Crear torneo", "Ver ranking"). El equipo debe discutir, estimar (con Planning Poker si es posible) y seleccionar las tareas para un "Sprint" de 15 minutos.

Desarrollo:

1. Asignación De Roles:

- Daniela Calderón: Producto Owner.
- Lenin Barrios: Scrum Master.
- Andres Garcia: Desarrollador Front-end
- Alberto Marquez: Desarrollador Back-end
- Jose Barbosa: Desarrollador Back-end y Tester

2. Product Backlog Simple:

- **Registro de usuario:** Los jugadores deben crear una cuenta para ver todo lo relacionado con el torneo.
- **Perfil de usuario:** Los jugadores deben poder ver y editar su perfil, incluyendo nombre, foto, estadísticas de torneos anteriores
- **Crear Torneo:** Función para que los usuarios puedan crear y configurar sus torneos, como también las reglas de estos.
- **Inscripción de jugadores:** Permitir a los jugadores registrarse para participar en los torneos.
- **Ver ranking:** Visualizar las posiciones de los jugadores o equipos en el torneo.
- **Resultados del torneo:** Calcular y mostrar los resultados finales de cada torneo.
- **Historial:** Mostrar a los jugadores un historial de los torneos en los que han participado, incluyendo sus posiciones y resultados, así como a los organizadores, el historial de los torneos que han creado, incluyendo el número de participantes, resultados y estado del torneo.

3. Sprint:

El equipo debate y selecciona las tareas que se abordarán durante el Sprint de 15 minutos. Cada tarea debe ser estimada utilizando Planning Poker si es posible

Duración del Sprint: 15 minutos

- **Planificación:**

Tareas: Definición de roles, Presentación del Product Backlog (registro de usuario, perfil de usuario, creación de torneo, inscripción de jugadores, ver ranking, resultados, historial), Estimación de tareas (con Planning Poker si es posible), Selección de tareas para el Sprint.

Herramientas: Gestión (Jira), Comunicación (Teams), Herramientas de estimación (Planning Poker).

Duración: 1 Sprint

- **Desarrollo Front-end:**

Tareas: Implementación de la interfaz gráfica de la página de registro de usuario, perfil de usuario, y vista de ranking usando React/HTML/CSS. Integración de la UI con el backend a través de API REST.

Herramientas: React, CSS, Git/GitHub para control de versiones, Postman para pruebas de API.

Duración: 1 Sprint

- **Desarrollo Back-end:**

Tareas: Desarrollo de la API REST para el registro de usuarios, creación de torneos, inscripción de jugadores y visualización de resultados.

Implementación de base de datos para almacenamiento de usuarios y torneos (PostgreSQL).

Herramientas: Node.js, Express, PostgreSQL, Git/GitHub, Postman (pruebas de API).

Duración: 1 Sprint

- **Pruebas y Ajustes:**

Tareas: Documentación del código y del sistema, preparación de la documentación para la entrega, despliegue de la aplicación en ambiente de producción.

Herramientas: Git/GitHub, Heroku o servidor similar para despliegue.

Duración: 1 Sprint

Actividad 3: Debate sobre Prácticas de XP 🗣️

Objetivo: Analizar la viabilidad de las prácticas de XP.

Temas de debate:

- **¿Es el "Pair Programming" siempre más eficiente que programar solo? ¿En qué escenarios sí y en cuáles no?**

R/ Para poder responder a esta pregunta decidimos dirigirnos en primera instancia a la definición de "pair programming", encontramos que es una técnica de desarrollo ágil en el que dos programadores trabajan en un solo puesto de trabajo; cada desarrollador cuenta con un rol: el primero es el conductor, quien se encarga de escribir el código mientras que el navegador revisa cada línea de código escrito, estos roles no son permanentes, sino que se van cambiando continuamente durante cada sesión.

Esta técnica cuenta con varias ventajas, entre ellas podemos encontrar: la calidad del código puede mejorar ya que la constante revisión de éste reduce los errores; gracias al trabajo en equipo es posible compartir diferentes técnicas, habilidades y estilos de programación; el diseño puede ser más sólido ya que son dos personas las que aportan ideas, conocimiento, alternativas que llevan a brindar soluciones bien analizadas y no impulsivas; otra ventana importante es que la responsabilidad no recae en una sola persona, no es este único desarrollador que conoce el código, son dos que pueden comprenderlo y de ser necesario modificarlo. Sin embargo también resulta importante resaltar las desventajas que presenta trabajar con el "pair programming": el hecho de que trabajen dos personas aumenta el esfuerzo total de la labor incrementando el costo de horas-personas; es necesario mantener la concentración y atención compartida, lo cual suele ser muy agotador durante largos periodos de tiempo; si la tarea es sencilla o muy rutinaria el trabajo en pareja puede ser excesivo; también pueden existir problemas de coordinación en el equipo y por último, es posible que sea difícil iniciar si ambos programadores cuentan con técnicas, habilidades y estilos diferentes de programación.

Luego de analizar toda esta información llegamos a la conclusión que el "pair programming" no es siempre más eficiente que trabajar solo, hay escenarios en los que sí resulta ser muy útil mientras que hay otros en los que no, por ejemplo: si la

tarea es desarrollar el módulo de autenticación de usuarios en una aplicación bancaria esta técnica lleva a una revisión constante del código, evitando así posibles errores de seguridad, mientras que el conductor escribe el código el navegador puede revisar y proponer mejoras al trabajo que hace; pero si vemos un caso en el que una empresa debe entregar una versión funcional de su sistema de inventario en dos días y se tienen dos programadores con técnicas y estilos diferentes que además no hayan trabajado nunca con pair programming la solución de la tarea será lenta e incluso puede que no lo lleguen a entregar en la fecha indicada.

En conclusión el pair programming no es eficiente siempre: cuando se deben realizar tareas simples la mejor opción es programar solo, en caso de que sean tareas críticas y complejas el pair programming es una muy buena opción a tener en cuenta; también es importante tener en cuenta otros factores a la hora de definir si desarrollar solo o con la técnica vista: el plazo de entrega, el conocimiento y la experiencia de los programadores, la familiaridad del trabajo en pareja, entre otros factores.

- **¿Es realista aplicar TDD en proyectos con fechas de entrega muy ajustadas?**

R/ El desarrollo guiado por pruebas es una metodología que se basa en realizar pruebas unitarias antes de escribir código. Se crean casos de prueba para las funcionalidades, luego se escribe el código y se testea y si se encuentra algún error, se reescribe el código. El objetivo del TDD es reducir errores desde el inicio mejorando así la calidad del software.

El implementar esta técnica conlleva ventajas como reducir los defectos desde el principio de la creación del software pero también conlleva ciertas contras como por ejemplo el esfuerzo y tiempo inicial que debe ser invertido para escribir y desarrollar estas pruebas, sobre todo si los programadores no cuentan con experiencia previa con esta metodología lo que no mejora la productividad al principio sino que lleva a que se consuma mucho tiempo en la fase inicial del proyecto.

Creemos que no siempre es realista implementar TDD en proyectos con plazos muy ajustados, ya que invertir mucho tiempo al inicio realizando pruebas antes de codificar puede llegar a ser contraproducente para la entrega final. Sin embargo, una implementación de TDD es posible si los miembros del equipo ya han trabajado de esta manera anteriormente, tienen un largo periodo de tiempo para entregar el

proyecto; aplicar TDD parcialmente también aumentaría la viabilidad, es decir, solamente en componentes principales y críticos en dónde sería de gran utilidad detectar Bugs desde el principio porque hacerlo después costaría tiempo y recursos.

- **¿Qué desafíos culturales podría enfrentar un equipo al intentar adoptar la propiedad colectiva del código?**

R/ La propiedad colectiva del código es una práctica en la que todos los desarrolladores del equipo tienen responsabilidad sobre cada módulo del proyecto, pueden y deben realizar cambios en cada uno, aportar correcciones, etc.

Esto puede llegar a tener varios pros, uno de esto es que un módulo no depende de un solo desarrollador y si esta falta, otro puede trabajar con ese módulo con facilidad; sin embargo, nos encontramos con varios retos culturales:

Algunos desarrolladores tienden a apegarse a un módulo en específico lo que dificulta la intervención de otros en éste.

También puede suceder que haya falta de responsabilidad ya que todos trabajan en todos los módulos, ninguno se siente responsable de un módulo y esto puede generar descuidos.

Si no se establecen reglas y buenas prácticas de codificación desde el inicio se pueden presentar inconsistencias, errores y duplicados en el código.

En caso de poca comunicación, de no compartir ideas sobre modificaciones en el código resultarán en códigos duplicados, en módulos que no funcionen o que no se acoplen al sistema.

Si algo no funciona, es probable que los desarrolladores empiecen a buscar el culpable dentro de su equipo, en lugar de reflexionar y buscar soluciones útiles para todos.